

MySQL Capstone Project

Database design for a retail company's online sales data across multiple Indian cities. The company wants to analyze:

Business Scenario

A retail company sells products online across multiple Indian cities. The company wants to analyze:

- Customers
- Products
- Orders
- Payments
- Delivery status
- Revenue
- Customer behavior

The goal is to build a MySQL database and write analytical SQL queries for management reporting.

Required Tables

1. customers

```
CREATE TABLE customers
(
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100),
  city VARCHAR(50),
  state VARCHAR(50),
  gender VARCHAR(10),
  membership_type VARCHAR(30)
);
```

2. products

```
CREATE TABLE products
(
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2)
);
```

3. orders

```
CREATE TABLE orders
(
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    order_status VARCHAR(30)
);
```

4. order_items

```
CREATE TABLE order_items
(
    item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT
);
```

5. payments

```
CREATE TABLE payments
(
    payment_id INT PRIMARY KEY,
    order_id INT,
```

```
payment_mode VARCHAR(30),  
payment_status VARCHAR(30),  
amount DECIMAL(10,2)  
);
```

6. deliveries

```
CREATE TABLE deliveries  
(  
    delivery_id INT PRIMARY KEY,  
    order_id INT,  
    delivery_partner VARCHAR(50),  
    delivery_status VARCHAR(30),  
    delivery_city VARCHAR(50)  
);
```

Capstone Requirements

Part 1: Database Design

Create database:

```
CREATE DATABASE retail_capstone_db;  
USE retail_capstone_db;
```

Create all 6 tables.

Add appropriate primary keys.

Optional advanced task:

Add foreign keys between:

```
orders.customer_id → customers.customer_id  
order_items.order_id → orders.order_id  
order_items.product_id → products.product_id  
payments.order_id → orders.order_id  
deliveries.order_id → orders.order_id
```

Part 2: Data Insertion

Insert at least:

Table	Minimum Records
customers	10
products	10
orders	15
order_items	20
payments	15
deliveries	15

Include some realistic data:

- Multiple customers from same city
- Multiple products from same category
- Customers with multiple orders
- Orders with multiple products
- Failed payments
- Pending deliveries
- Cancelled orders

Part 3: Basic Queries

Write SQL queries for:

1. Display all customers.
2. Display customer name, city, and membership type.
3. Display all products sorted by price descending.
4. Find customers from Hyderabad.
5. Find Gold membership customers.
6. Find products priced between ₹500 and ₹5000.
7. Find products from categories Electronics and Fashion.
8. Find orders placed after 2026-01-01 .
9. Find payments made using UPI.

10. Find deliveries that are still pending.

Part 4: Aggregate Queries

Write SQL queries for:

11. Count total customers.
 12. Count total orders.
 13. Count total products.
 14. Find total revenue from successful payments.
 15. Find average order payment amount.
 16. Find highest payment amount.
 17. Find lowest payment amount.
 18. Count customers by city.
 19. Count products by category.
 20. Count orders by order status.
-

Part 5: Joins

Write SQL queries for:

21. Display customer name with order ID and order date.
22. Display order ID, product name, quantity, and price.
23. Display customer name, product name, quantity, and order date.
24. Display order ID with payment mode, payment status, and amount.
25. Display order ID with delivery partner and delivery status.
26. Display full order report:

Customer Name
City
Order ID
Order Date
Product Name
Category
Quantity
Price
Payment Status
Delivery Status

Part 6: GROUP BY and HAVING

Write SQL queries for:

- 27. Total revenue by city.
 - 28. Total revenue by customer.
 - 29. Total quantity sold by product.
 - 30. Revenue by product category.
 - 31. Number of orders by customer.
 - 32. Customers having more than 1 order.
 - 33. Product categories having revenue greater than ₹10,000.
 - 34. Cities having more than 2 customers.
 - 35. Products sold more than 3 times.
-

Part 7: Subqueries

Write SQL queries for:

- 36. Find customers who placed orders.
 - 37. Find customers who never placed orders.
 - 38. Find products that were never ordered.
 - 39. Find orders with payment amount greater than average payment amount.
 - 40. Find customer who made the highest payment.
 - 41. Find products priced above the average product price.
 - 42. Find customers who ordered Electronics products.
 - 43. Find orders that have successful payments.
 - 44. Find orders that are not delivered yet.
 - 45. Find customers whose total spending is above average customer spending.
-

Part 8: Data Quality Checks

Write SQL queries for:

- 46. Find orders without payment records.
- 47. Find orders without delivery records.
- 48. Find payments where amount is NULL or 0.

49. Find cancelled orders with successful payment.
 50. Find delivered orders with failed payment.
 51. Find order items with invalid product IDs.
 52. Find orders with invalid customer IDs.
-
-

Submission Requirements

Each cohort should submit:

1. SQL file containing table creation scripts
 2. SQL file containing insert statements
 3. SQL file containing all query solutions
 4. Short document explaining:
 - Database design
 - Table relationships
 - Key insights from reports
-